上位机配合

嵌入式设备应用开发（上位机配合）

原创

嵌入式-老费 已于 2023-08-22 12:31:51 修改

阅读量1.2k 收藏 2

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 嵌入式硬件

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 本文介绍了嵌入式设备与上位机配合的重要性，并推荐使用C# & WPF进行上位机开发，因为其语法简洁且界面绘制方便。详细阐述了上位机和嵌入式设备的通信方式，如232、485、CAN、USB和以太网，并指导如何创建C# & WPF工程，分析关键代码文件。最后，通过添加进度圈功能展示上位机开发的基本过程。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing @163.com】

很多的嵌入式设备，其实是需要和上位机配合使用的。简单的配置下ip、port等参数就可以用起来了。但是对于复杂的嵌入式设备，除了基本的网络配置之外，算法参数、操作流程、业务逻辑、标定方法，这些都需要上位机深度参与的。简单的配置，是无法让嵌入式设备正常工作的。

目前上位机开发的方法很多。早期的mfc，后来的c# form、qt、python qt等等，其实都可以拿来进行上位机开发。这里，我特别推荐c# & wfc的方法来进行上位机开发。c# 不仅仅语法比较简单，容易上手，界面绘制也全部是用脚本完成的。用一个不太恰当的比方，整个上位机就像是在开发网站，脚本和界面相当于前端开发，而c#则相当于后端开发。

1、上位机和嵌入式通信的方式

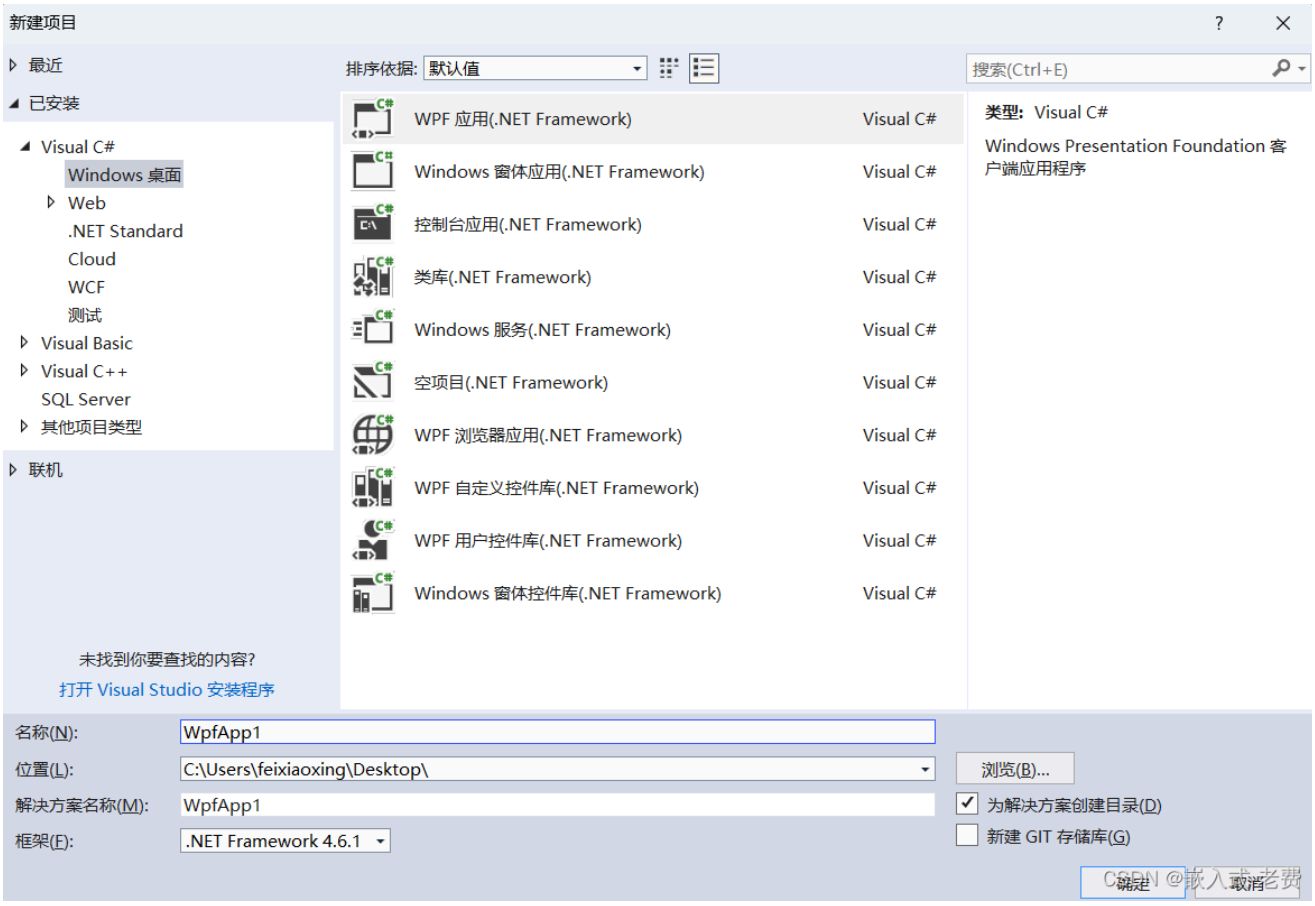
主要就是232、485、can、usb和以太网，其中usb稍微复杂点，一般需要libusb等库配合。

2、创建c# & wpf的前提

需要安装visual studio开发包，以及c#开发包。

3、创建c# & wpf工程

输入“文件”-》“新建”-》“项目”，选择“c#”-》“Windows 桌面”，选择WPF应用即可，



4、代码分析

整个工程比较简单，需要重点关注的其实就是三个文件。第一，App.xaml.cs文件，它负责整个app的初始化工作。当然，如果没有约定内容的话，这部分其实是空的，

```
using System;
using System.Collections.Generic;
using System.Configuration;
using System.Data;
using System.Linq;
using System.Threading.Tasks;
using System.Windows;

namespace WpfApp1
{
    /// <summary>
    /// App.xaml 的交互逻辑
    /// </summary>
    public partial class App : Application
    {
```

```
    }
}
```

第二，MainWindow.xaml.cs文件，它主要是MainWindow中处理和逻辑相关的内容。

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace WpfApp1
{
    /// <summary>
    /// MainWindow.xaml 的交互逻辑
    /// </summary>
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

第三，则是MainWindow.xaml，这部分主要是负责界面显示和设计。所用的编写语言有点类似于html，内容是这样的，

```
<Window x:Class="WpfApp1.MainWindow"
```

```
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        xmlns:local="clr-namespace:WpfApp1"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
   <Grid Width="120" Height="120">
</Grid>
</Window>
```

有这三部分，基本上就凑成了一个基本的app程序，但这是完全不够的。

5、添加一个进度圈功能

简单说这些代码还是很无聊、枯燥的。我们可以设计一个进度圈的功能。所有的操作只需要两个步骤，第一，就是添加一个Circle.cs文件，

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Media;
using System.Windows.Shapes;

namespace WpfApp1
{
    public class Circle : Shape
    {
        #region 成员变量
        private Geometry m_Data;
        private Geometry Data
        {
            get { return m_Data; }
            set
            {
                if (m_Data != value)
                {
```

```
                m_Data = value;
                this.InvalidateVisual();
            }
        }
    }

    private int m_Status;
    #endregion

    #region 重写方法
    /// <summary>
    /// 监听StrokeThickness变化
    /// </summary>
    /// <param name="e"></param>
    protected override void
OnPropertyChanged(DependencyPropertyChangedEventArgs e)
    {
        base.OnPropertyChanged(e);
        if (e.Property == StrokeThicknessProperty)
        {
            propertyChangedCallback(this, new
DependencyPropertyChangedEventArgs());
        }
        else if (e.Property == FillProperty)
        {
            m_Status = e.NewValue == null ? 1 : 2;
            propertyChangedCallback(this, new
DependencyPropertyChangedEventArgs());
        }
        else if (e.Property == WidthProperty || e.Property ==
HeightProperty || e.Property == ActualWidthProperty || e.Property ==
ActualHeightProperty)
        {
            propertyChangedCallback(this, new
DependencyPropertyChangedEventArgs());
        }
    }

    /// <summary>
    /// 重写此方法实现图形绘制
    /// </summary>
```

```
        protected override Geometry DefiningGeometry { get { return
Data; } }
        #endregion

        #region 依赖属性
        /// <summary>
        /// 起始角度
        /// </summary>
        public double StartAngle
        {
            get { return (double)GetValue(StartAngleProperty); }
            set { SetValue(StartAngleProperty, value); }
        }

        // Using a DependencyProperty as the backing store for
StartAngle.  This enables animation, styling, binding, etc...
        public static readonly DependencyProperty StartAngleProperty =
            DependencyProperty.Register("StartAngle", typeof(double),
typeof(Circle), new PropertyMetadata(0.0, propertyChangedCallback));

        /// <summary>
        /// 终止角度
        /// </summary>
        public double EndAngle
        {
            get { return (double)GetValue(EndAngleProperty); }
            set { SetValue(EndAngleProperty, value); }
        }

        // Using a DependencyProperty as the backing store for
EndAngle.  This enables animation, styling, binding, etc...
        public static readonly DependencyProperty EndAngleProperty =
            DependencyProperty.Register("EndAngle", typeof(double),
typeof(Circle), new PropertyMetadata(0.0, propertyChangedCallback));

        /// <summary>
        /// 内边距
        /// </summary>
        public double Padding
        {
            get { return (double)GetValue(PaddingProperty); }
```

```
            set { SetValue(PaddingProperty, value); }
        }

        //protected override Geometry DefiningGeometry => throw new
NotImplementedException();

        // Using a DependencyProperty as the backing store for Padding.
This enables animation, styling, binding, etc...
        public static readonly DependencyProperty PaddingProperty =
            DependencyProperty.Register("Padding", typeof(double),
typeof(Circle), new PropertyMetadata(0.0, propertyChangedCallback));

        #endregion

        #region 图形语句生成
        private static void propertyChangedCallback(DependencyObject d,
DependencyPropertyChangedEventArgs e)
        {
            if (d is Circle circle)
            {
                // 计算内边距
                var padding = circle.m_Status == 1 ?
circle.StrokeThickness / 2 : 0;
                padding += circle.Padding;

                // 判断绘制的圆弧是否可见，以提高性能
                double width = circle.Width;
                double height = circle.Height;
                if (double.IsNaN(width) || double.IsNaN(height))
                {
                    if (double.IsNaN(circle.DesiredSize.Width) ||
double.IsNaN(circle.DesiredSize.Height) || circle.DesiredSize == new
Size(0, 0))
                    {
                        width = circle.ActualWidth;
                        height = circle.ActualHeight;
                    }
                    else
                    {
                        width = circle.DesiredSize.Width;
                        height = circle.DesiredSize.Height;
```

```
                }
            }
            if (width > 0 && height > 0 && circle.StartAngle !=
circle.EndAngle)
            {
                double ellipseA = width / 2 - padding;
                double ellipseB = height / 2 - padding;

                // 起止角度间隔大于等于360°，直接画圆

                if (Math.Abs(circle.StartAngle - circle.EndAngle)
>= 360)
                {
                    string data = $"M{padding + ellipseA * 2},
{padding + ellipseB + 0.001} A{ellipseA},{ellipseB} 0,1,1 {padding +
ellipseA * 2},{padding + ellipseB}z";
                    circle.Data = (Geometry)Geometry.Parse(data);
                    return;
                }

                // 椭圆公式：X²/a²+Y²/b²=1
                // Rect与椭圆各参数的对应关系：
                // Rect.X与Rect.Y分别是椭圆外接矩形相对Circle区域的左
上角的偏移量;
                // Rect.Width与Rect.Height分别是椭圆外接矩形的宽和高
                // 以下根据StartAngle和EndAngle算出圆弧在矩形函数曲线
中的起始点和终止点

                // 判断绘制方向
                bool clockWise = circle.EndAngle >
circle.StartAngle;

                // 判断优弧/劣弧
                bool majorArc = Math.Abs(circle.StartAngle -
circle.EndAngle) % 360 >= 180;

                // 将起始点和终止点转化为椭圆上的坐标
                double rad = 0;

                Point startPoint = new Point();
                if ((90 - circle.StartAngle) % 360 == 0)
```

```
                    {
                        startPoint.X = 0;
                        startPoint.Y = ellipseB;
                    }
                    else if ((270 - circle.StartAngle) % 360 == 0)
                    {
                        startPoint.X = 0;
                        startPoint.Y = -ellipseB;
                    }
                    else
                    {
                        rad = GetRadian(circle.StartAngle);
                        startPoint.X = ellipseA * ellipseB /
Math.Sqrt(Math.Pow(ellipseB, 2) + Math.Pow(ellipseA * Math.Tan(rad),
2));
                        startPoint.X *= Math.Cos(rad) > 0 ? 1 : -1;
                        startPoint.Y = startPoint.X * Math.Tan(rad);
                    }

                    Point endPoint = new Point();
                    if ((90 - circle.EndAngle) % 360 == 0)
                    {
                        endPoint.X = 0;
                        endPoint.Y = ellipseB;
                    }
                    else if ((270 - circle.EndAngle) % 360 == 0)
                    {
                        endPoint.X = 0;
                        endPoint.Y = -ellipseB;
                    }
                    else
                    {
                        rad = GetRadian(circle.EndAngle);
                        endPoint.X = ellipseA * ellipseB /
Math.Sqrt(Math.Pow(ellipseB, 2) + Math.Pow(ellipseA * Math.Tan(rad),
2));
                        endPoint.X *= Math.Cos(rad) > 0 ? 1 : -1;
                        endPoint.Y = endPoint.X * Math.Tan(rad);
                    }

                    string pathData = $"M";
```

```
                    if (circle.m_Status == 2)
                    {
                        pathData += $"{ellipseA + padding},{ellipseB + 
padding} ";
                    }
                    pathData += $"{startPoint.X + ellipseA + padding}, 
{startPoint.Y + ellipseB + padding} ";
                    pathData += $"A{ellipseA},{ellipseB} ";
                    pathData += $"0,{(majorArc ? "1" : "0")}, 
{(clockWise ? "1" : "0")} ";
                    pathData += $"{endPoint.X + ellipseA + padding}, 
{endPoint.Y + ellipseB + padding}";
                    circle.Data = (Geometry)Geometry.Parse(pathData);
                }
                else
                {
                    circle.Data = (Geometry)Geometry.Parse("");
                }
            }
        }

        /// <summary>
        /// 将角度转化为弧度
        /// </summary>
        /// <param name="angle"></param>
        /// <returns></returns>
        private static double GetRadian(double angle)
        {
            return angle / 180.0 * Math.PI;
        }
        #endregion
    }
}
```

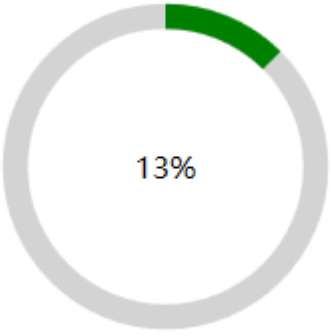
第二，就是更新下MainWindow.xaml文件，修改成这样，主要是增加Circle和Text的内容，

```
<Window x:Class="WpfApp1.MainWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        xmlns:local="clr-namespace:WpfApp1"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
   <Grid Width="120" Height="120">
    <local:Circle StartAngle="0" EndAngle="360" Stroke="LightGray"
StrokeThickness="10"/>
    <local:Circle StartAngle="-90" EndAngle="-45" Stroke="Green"
StrokeThickness="10"/>
    <TextBlock Text="13%" HorizontalAlignment="Center"
VerticalAlignment="Center"/>
</Grid>
</Window>
```

这样编辑完成后，我们编译一下，就可以实现如下这样的效果。大家看下，是不是很神奇，



CSDN @嵌入式-老费